COSC2637 Assignment 3 - a3.scala source code (s4271512)

===== a3.scala =====

```
// Assignment 3 - Query Log Analysis Using Spark Streaming (COSC2637, PG)
// Run:    spark-submit --class streaming.a3 --master yarn --deploy-mode client a3.jar
//           hdfs:///input hdfs:///output 250
// Input:  query log CSV files arriving in the monitored HDFS directory (args(0))
//         query_id,user_id,query_text,event_time,device,result_count,response_time_ms
// Output: args(1)/task1-NNN, task2-NNN, task3-NNN with NNN = 001, 002, ... per task;
//         a folder is only written for a batch whose RDD is not empty.
// args(2) is the slow-query threshold in ms. Batch interval: 3 seconds.
//
// All three tasks work on the same DStream. Each batch is parsed and cleaned once,
// Task 1 counts the cleaned queries, Task 2 summarises the slow queries and Task 3
// adds the Task 1 counts to the running state kept by updateStateByKey.

package streaming

import java.util.Locale

import scala.collection.mutable
import scala.math.BigDecimal.RoundingMode
import scala.util.Try

import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{LongWritable, Text}
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat
import org.apache.spark.SparkConf
import org.apache.spark.streaming.dstream.DStream
import org.apache.spark.streaming.{Seconds, StreamingContext}

object a3 {

  val BatchInterval = Seconds(3)
  val CheckpointDir = "/s4271512/checkpoint"
  // The state is checkpointed every 5 batches (5-10 x the batch interval is the usual
  // rule): often enough to cut its growing lineage, not so often that it slows batches.
  val StateCheckpointInterval = Seconds(15)

  val StopWords = Set("the", "and", "for", "with", "how", "what", "why", "can", "you", "use", "using")

  // One valid input line after text processing.
  case class QueryRecord(userId: String, cleanedQuery: String, responseTimeMs: Long)

  // Partial slow-query result for one cleaned query, merged by reduceByKey.
  case class SlowAggregate(count: Long, totalResponseMs: Long, maxResponseMs: Long, users: Set[String])

  // One Task 2 output row.
  case class SlowQueryIncident(cleanedQuery: String,
                               slowQueryCount: Long,
                               maxResponseMs: Long,
                               averageResponseMs: BigDecimal,
                               affectedUsers: Int,
                               severityLevel: String) {
    def toCsv: String =
      Seq(cleanedQuery, slowQueryCount, maxResponseMs, averageResponseMs.bigDecimal.toPlainString,
        affectedUsers, severityLevel).mkString(",")
  }

  // Task 3 state of one cleaned query, kept across all batches.
  case class QueryStats(currentBatchCount: Long,
                        previousAccumulatedCount: Long,
                        accumulatedCount: Long,
                        countChange: Long,
                        trendStatus: String) {
    def toCsv(cleanedQuery: String): String =
      Seq(cleanedQuery, currentBatchCount, previousAccumulatedCount, accumulatedCount,
        countChange, trendStatus).mkString(",")
  }

  // Task 3 ranking: accumulated_count DESC, current_batch_count DESC, cleaned_query ASC.
  val TrendingOrder: Ordering[(String, QueryStats)] =
    Ordering.by { case (query, stats) => (-stats.accumulatedCount, -stats.currentBatchCount, query) }

  // Output folder number of each task. foreachRDD bodies run on the driver, so a plain
  // counter works; it only goes up when a folder is actually written.
```

```
val outputSequence = mutable.Map("task1" -> 0, "task2" -> 0, "task3" -> 0)

def main(args: Array[String]): Unit = {
  if (args.length != 3) {
    System.err.println("Usage: streaming.a3 <monitored input dir> <output dir> <slow query threshold ms>")
    System.exit(1)
  }
  val inputDir = args(0)
  val outputDir = args(1).stripSuffix("/")
  val threshold = Try(args(2).trim.toDouble).getOrElse {
    System.err.println(s"Slow query threshold must be a number of milliseconds, got: ${args(2)}")
    sys.exit(1)
  }

  val conf = new SparkConf()
    .setAppName("A3 Query Log Analysis (s4271512)")
    .set("spark.streaming.stopGracefullyOnShutdown", "true") // finish the running batch on Ctrl+C
  val ssc = new StreamingContext(conf, BatchInterval)
  ssc.sparkContext.setLogLevel("WARN")
  // updateStateByKey needs a checkpoint directory to save the state RDDs.
  ssc.checkpoint(CheckpointDir)

  // 1. Input: only files that arrive after the job started are read. fileStream is used
  //    instead of textFileStream so the filter can skip "<name>._COPYING_", the temporary
  //    file `hadoop fs -put` writes before renaming it. Otherwise a file could be counted twice.
  val lines: DStream[String] = ssc
    .fileStream[LongWritable, Text, TextInputFormat](inputDir, isQueryLogFile _, newFilesOnly = true)
    .map { case (_, line) => line.toString }

  // 2. Text processing. The header line, malformed lines and queries that are empty after
  //    cleaning are dropped here. Cached because Task 1 and Task 2 both read it.
  val records: DStream[QueryRecord] = lines.flatMap(line => parseRecord(line))
  records.cache()

  // 3. Task 1: count of each cleaned query in the current batch.
  //    batchCounts is cached because it is also the input of Task 3.
  val batchCounts: DStream[(String, Long)] = records
    .map(record => (record.cleanedQuery, 1L))
    .reduceByKey(_ + _)
  batchCounts.cache()

  //    sortBy range-partitions the rows, so part-00000, part-00001, ... each hold one
  //    contiguous range and task1-NNN/* read in file order is globally sorted.
  //    No single partition has to take all the data.
  batchCounts.foreachRDD { rdd =>
    if (!rdd.isEmpty()) {
      rdd.sortBy { case (query, count) => (-count, query) } // count DESC, query ASC
        .map { case (query, count) => s"$query,$count" }
        .saveAsTextFile(nextOutputPath(outputDir, "task1"))
    }
  }

  // 4. Task 2: slow queries (response_time_ms > threshold) of the current batch, one row per
  //    cleaned query. reduceByKey combines on the map side first, so each partition sends
  //    only one partial result per query into the shuffle.
  val slowIncidents: DStream[SlowQueryIncident] = records
    .filter(_.responseTimeMs > threshold)
    .map(r => (r.cleanedQuery, SlowAggregate(1L, r.responseTimeMs, r.responseTimeMs, Set(r.userId))))
    .reduceByKey(mergeSlowAggregates)
    .map { case (query, agg) => toIncident(query, agg, threshold) }

  //    severity_level ASC, max_response_time_ms DESC, slow_query_count DESC, cleaned_query ASC.
  //    "CRITICAL" < "WARNING" as text, so CRITICAL rows come first.
  slowIncidents.foreachRDD { rdd =>
    if (!rdd.isEmpty()) {
      rdd.sortBy(i => (i.severityLevel, -i.maxResponseMs, -i.slowQueryCount, i.cleanedQuery))
        .map(_.toCsv)
        .saveAsTextFile(nextOutputPath(outputDir, "task2"))
    }
  }

  // 5. Task 3: the Task 1 counts flow into updateStateByKey, which keeps one QueryStats per
  //    cleaned query since the job started.
  val queryStats: DStream[(String, QueryStats)] = batchCounts.updateStateByKey(updateQueryStats _)
  queryStats.checkpoint(StateCheckpointInterval)

  //    PG requirement: only write when new query log data arrived in this batch. A query has
```

```
  //    currentBatchCount > 0 only if it was in this batch's new files, so in an idle batch
  //    the filter is empty and nothing is written.
  //    takeOrdered keeps a top 10 per partition and merges them, so the whole state is never
  //    sorted; only those 10 rows are written.
  queryStats.foreachRDD { rdd =>
    val receivedNewData = !rdd.filter { case (_, stats) => stats.currentBatchCount > 0 }.isEmpty()
    if (receivedNewData) {
      val top10 = rdd.takeOrdered(10)(TrendingOrder)
      rdd.sparkContext
        .parallelize(top10.map { case (query, stats) => stats.toCsv(query) }, numSlices = 1)
        .saveAsTextFile(nextOutputPath(outputDir, "task3"))
    }
  }

  ssc.start()
  ssc.awaitTermination()
}

// Accept finished data files only: no hidden files and no `hadoop fs -put` copy in progress.
def isQueryLogFile(path: Path): Boolean = {
  val name = path.getName
  !name.startsWith(".") && !name.startsWith("_") && !name.endsWith("._COPYING_")
}

// query_text is everything between the 2nd field and the last 4 fields, so a comma inside
// the query text does not shift the other columns. The header and malformed lines have no
// numeric response_time_ms and return None, as does a query that is empty after cleaning.
def parseRecord(line: String): Option[QueryRecord] = {
  val fields = line.split(",", -1)
  if (fields.length < 7) {
    None
  } else {
    val n = fields.length
    val responseTimeMs = Try(fields(n - 1).trim.toLong).toOption
    val cleanedQuery = cleanQuery(fields.slice(2, n - 4).mkString(","))
    responseTimeMs
      .filter(_ => cleanedQuery.nonEmpty)
      .map(ms => QueryRecord(fields(1).trim, cleanedQuery, ms))
  }
}

// Text processing steps 1-7 from the specification. Returns "" if no term is left.
def cleanQuery(queryText: String): String =
  queryText
    .toLowerCase(Locale.ROOT)        // 1. case folding
    .replaceAll("[^a-z0-9\\s]", "")  // 2. remove non-alphanumeric characters (spaces stay)
    .trim                            // 3. remove extra spaces (outer ones here, inner by the split)
    .split("\\s+")                   // 4. split the query into terms
    .filter(_.length >= 3)           // 5. remove terms shorter than 3 characters
    .filterNot(StopWords.contains)   // 6. remove stop words
    .mkString(" ")                   // 7. rebuild the cleaned query

def mergeSlowAggregates(a: SlowAggregate, b: SlowAggregate): SlowAggregate =
  SlowAggregate(
    a.count + b.count,
    a.totalResponseMs + b.totalResponseMs,
    math.max(a.maxResponseMs, b.maxResponseMs),
    a.users ++ b.users) // distinct users of this query in the batch

// The average is computed from the exact sum with BigDecimal and rounded half-up to 2
// decimals, so the result does not depend on how Spark split the data.
def toIncident(query: String, agg: SlowAggregate, threshold: Double): SlowQueryIncident = {
  val average = (BigDecimal(agg.totalResponseMs) / agg.count).setScale(2, RoundingMode.HALF_UP)
  val severity = if (agg.maxResponseMs >= threshold * 2) "CRITICAL" else "WARNING"
  SlowQueryIncident(query, agg.count, agg.maxResponseMs, average, agg.users.size, severity)
}

// updateStateByKey runs this for every query in the state or in the batch. newCounts holds
// the Task 1 count of this batch and is empty if the query did not appear in it.
//   NEW       - no state yet, first time the query is seen
//   INCREASED - known query that appeared again in this batch
//   UNCHANGED - known query that did not appear in this batch
def updateQueryStats(newCounts: Seq[Long], state: Option[QueryStats]): Option[QueryStats] = {
  val currentBatchCount = newCounts.sum
  val previousAccumulated = state.map(_.accumulatedCount).getOrElse(0L)
  val accumulated = previousAccumulated + currentBatchCount
  val trendStatus =
```

```
      if (state.isEmpty) "NEW"
      else if (currentBatchCount > 0) "INCREASED"
      else "UNCHANGED"
    Some(QueryStats(currentBatchCount, previousAccumulated, accumulated,
      accumulated - previousAccumulated, trendStatus))
  }

  // Next output folder of a task: <output>/task1-001, task1-002, ...
  def nextOutputPath(outputDir: String, task: String): String = {
    val next = outputSequence(task) + 1
    outputSequence(task) = next
    f"$outputDir/$task-$next%03d"
  }
}
```